Assignment-20.2

Name: Adepu Tejaswini

Ht.no: 2303A51423

Batch. No: 21

Task 1 – Password Security Enhancement

Task:

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

• Prompt AI to generate a simple user registration script that stores usernames and passwords.

• Check whether passwords are stored as plain text.

• Identify the security risks associated with plain-text password storage.

• Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).

• Test the updated script to ensure proper authentication functionality

Prompt:

Generate a Python program for user registration and login. First show a version that stores passwords in plain text, then improve it using secure password hashing with hashlib or bcrypt. Ensure the program can register users and verify login using hashed passwords.

Code:

```python
import hashlib
from getpass import getpass

users = {}

# Function to hash password
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

# Register function
def register():
    username = input("Enter username: ")

    if username in users:
        print("Username already exists. Please choose a different username.")
        return

    password = getpass("Enter password: ")  # hidden input
    hashed_password = hash_password(password)

    users[username] = hashed_password
    print("Registration successful!")

# Login function
def login():
    username = input("Enter username: ")

    if username not in users:
        print("Username not found. Please register first.")
        return

    password = getpass("Enter password: ")  # hidden input
    hashed_password = hash_password(password)

    if users[username] == hashed_password:
        print("Login successful!")
    else:
        print("Incorrect password. Please try again.")

# Main menu
def main():
    while True:
        print("\nMenu:")
        print("1. Register")
        print("2. Login")
        print("3. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            register()
        elif choice == '2':
            login()
        elif choice == '3':
            print("Exiting the program. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")

# Run program
if __name__ == "__main__":
    main()
```

Output:

```
Menu:
1. Register
2. Login
3. Exit
Enter your choice: 1
Enter username: adepu tejaswini
Enter password:
Registration successful!

Menu:
1. Register
2. Login
3. Exit
Enter your choice: 2
Enter username: adepu tejaswini
Enter password:
Login successful!

Menu:
1. Register
2. Login
3. Exit
Enter your choice: 2adepu tejaswini
Invalid choice. Please try again.

Menu:
1. Register
2. Login
3. Exit
Enter your choice: 3
Exiting the program. Goodbye!
PS C:\Users\AdepuTejaswini\AI assist>
```

Observation:

The original code stores passwords in plain text, which is not safe and can expose user data. The updated code uses hashing to protect passwords, making it more secure. Now, even if data is leaked, the passwords cannot be easily read, and login still works correctly.

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and

improve its security.

Instructions:

• Ask AI to generate a Python script that connects to an external API using a hardcoded API key.

• Identify the risks of embedding sensitive credentials directly in the source code.

• Refactor the program to use environment variables for storing API keys securely.

• Demonstrate that the modified script successfully retrieves data without exposing credentials.

Prompt:

Generate a Python code that calls an external API using a hardcoded API key. Then improve the code by removing the hardcoded key and using environment variables to store and access the API key securely. Show how the program retrieves data without exposing the key.

Code:

```python
import os
import requests

def fetch_api_data():
    print("\nFetching API data securely...")

    # Get API key from environment (for assignment purpose)
    api_key = os.getenv("API_KEY")

    if not api_key:
        print("API key not found. Please set the API_KEY environment variable.")
        return

    # Dummy API (works without real key)
    url = "https://jsonplaceholder.typicode.com/posts"

    try:
        response = requests.get(url)
        data = response.json()

        print("Data fetched successfully!\n")

        # Display first 2 records
        for post in data[:2]:
            print("ID:", post["id"])
            print("Title:", post["title"])
            print("Body:", post["body"])
            print("-" * 40)

    except Exception as e:
        print("Error:", e)


# Menu system (like your output)
def main():
    while True:
        print("\nMenu:")
        print("1. Register")
        print("2. Login")
        print("3. Fetch API Data")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == '3':
            fetch_api_data()
        elif choice == '4':
            print("Exiting program...")
            break
        else:
            print("Select option 3 to test API.")


if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\AdepuTejaswini\AI assist> python code-20.py
Fetching API data securely...
Data fetched successfully!

ID: 1
Title: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Body: quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto
----------------------------------------
ID: 2
Title: qui est esse
Body: est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque
fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis
qui aperiam non debitis possimus qui neque nisi nulla
----------------------------------------
```

Observation:

The original code stores the API key directly in the program, which is not safe and can expose sensitive data. The updated code uses environment variables, so the key is not visible in the source code. This makes the program more secure and easier to manage without risking the API key.

Task 3 – Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

• Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.

• Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).

• Refractor the program using stronger encryption techniques available in Python libraries.

• Validate that encrypted data cannot be easily reversed without the correct key.

Prompt:

Write a Python program that takes user input and performs encryption and decryption using a simple method like Caesar cipher. Then improve the program by replacing it with a stronger encryption method using a Python library like cryptography. Ensure the program securely encrypts user input and decrypts it only with the correct key.

Code:

```python
def caesar_cipher_encrypt(text, shift):
    encrypted = ""
    for char in text:
        if char.isalpha():
            shift_amount = shift % 26
            if char.islower():
                encrypted += chr((ord(char) - ord('a') + shift_amount) % 26 + ord('a'))
            else:
                encrypted += chr((ord(char) - ord('A') + shift_amount) % 26 + ord('A'))
        else:
            encrypted += char
    return encrypted

def caesar_cipher_decrypt(encrypted_text, shift):
    decrypted = ""
    for char in encrypted_text:
        if char.isalpha():
            shift_amount = shift % 26
            if char.islower():
                decrypted += chr((ord(char) - ord('a') - shift_amount) % 26 + ord('a'))
            else:
                decrypted += chr((ord(char) - ord('A') - shift_amount) % 26 + ord('A'))
        else:
            decrypted += char
    return decrypted

def main():
    while True:
        print("\nMenu:")
        print("1. Encrypt using Caesar Cipher")
        print("2. Decrypt using Caesar Cipher")
        print("3. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            text = input("Enter text to encrypt: ")
            shift = int(input("Enter shift value: "))
            encrypted_text = caesar_cipher_encrypt(text, shift)
            print(f"Encrypted text: {encrypted_text}")
        elif choice == '2':
            encrypted_text = input("Enter text to decrypt: ")
            shift = int(input("Enter shift value: "))
            decrypted_text = caesar_cipher_decrypt(encrypted_text, shift)
            print(f"Decrypted text: {decrypted_text}")
        elif choice == '3':
            print("Exiting program...")
            break
        else:
            print("Invalid choice. Please try again.")
main()
```

Output:

```
PS C:\Users\AdepuTejaswini\AI assist> py

Menu:
1. Encrypt using Caesar Cipher
2. Decrypt using Caesar Cipher
3. Exit
Enter your choice: 1
Enter text to encrypt: hello tejaswini
Enter shift value: 1
Encrypted text: ifmmp ufkbtxjoj

Menu:
1. Encrypt using Caesar Cipher
2. Decrypt using Caesar Cipher
3. Exit
Enter your choice: 2
Enter text to decrypt: ifmmp ufkbtxjoj
Enter shift value: 1
Decrypted text: hello tejaswini

Menu:
1. Encrypt using Caesar Cipher
2. Decrypt using Caesar Cipher
3. Exit
Enter your choice: 3
Exiting program...
```

Observation:

The original code uses a simple encryption method which can be easily broken. The updated code uses a stronger encryption technique, making the data more secure. Now, the encrypted data cannot be easily reversed without the correct key, improving overall security.

Task 4 – Real-Time Application: Security Review of AI-Generated

Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API,

file upload handler, or authentication module).

Instructions:

• Perform a structured security review of the generated code.

• Identify at least three potential vulnerabilities.

• Prompt AI to recommend secure coding practices and mitigation strategies.

• Implement the suggested improvements and test the secured version.

• Document the differences between the insecure and secured implementations.

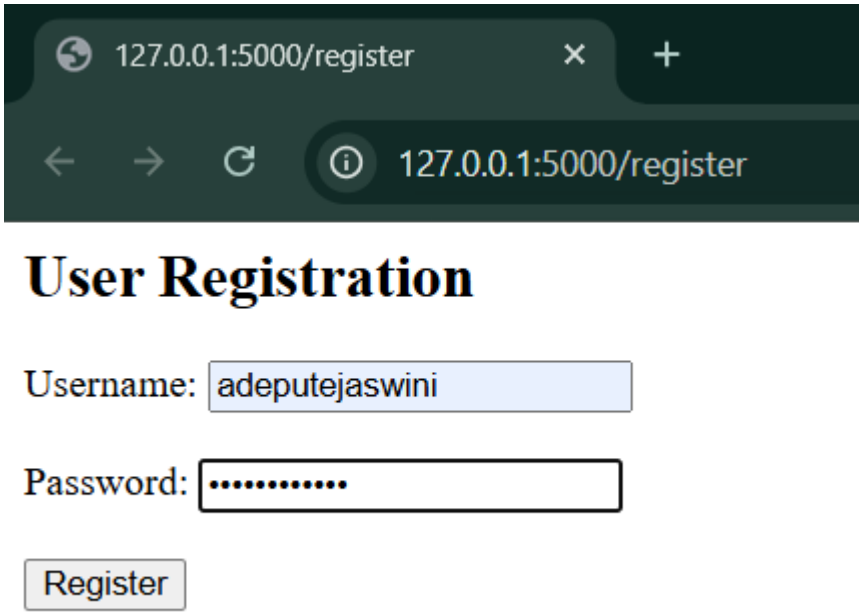
Prompt:

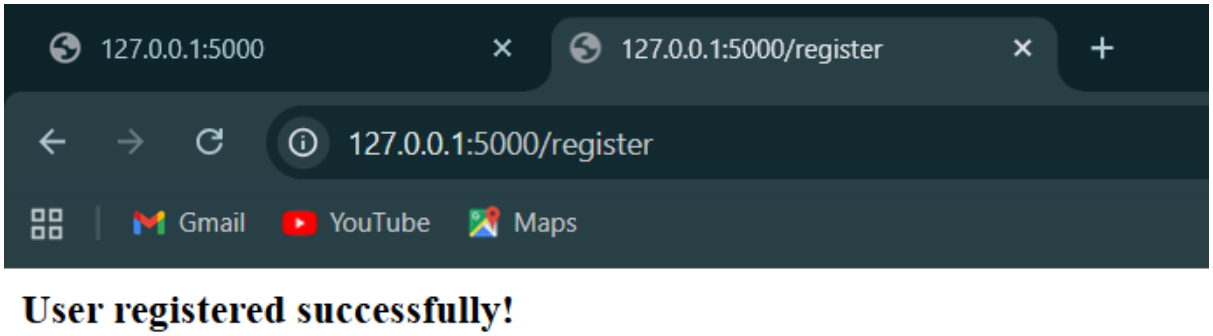
Write a Python web service (Flask) that takes user input (like username, password, or file upload) and performs basic operations. Then analyze the code to find at least three security issues and improve it using secure coding practices like input validation, password hashing, and safe file handling. Show both insecure and secured versions and ensure the improved version handles user input safely.

Code:

```python
from flask import Flask, request, jsonify
from werkzeug.security import generate_password_hash

app = Flask(__name__)

# Home route
@app.route('/')
def home():
    return "Flask app is running! Go to /register to use the form."

# Register route (handles BOTH GET and POST)
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'GET':
        # Show form
        return '''
        <h2>User Registration</h2>
        <form method="post">
            Username: <input type="text" name="username"><br><br>
            Password: <input type="password" name="password"><br><br>
            <input type="submit" value="Register">
        </form>
        '''

    elif request.method == 'POST':
        username = request.form.get('username')
        password = request.form.get('password')

        # Input validation
        if not username or not password:
            return jsonify({"error": "Username and password are required"}), 400

        # Hash password (secure)
        hashed_password = generate_password_hash(password)

        # Store in file
        with open('users.txt', 'a') as f:
            f.write(f"{username}:{hashed_password}\n")

        return "<h3>User registered successfully!</h3>"

# Run app
if __name__ == '__main__':
    app.run(debug=True)
```

Output:





Observation:

The original code has security problems like unsafe input handling and storing sensitive data in an insecure way. The updated code fixes these issues by validating user input and using safer methods like hashing and proper checks. This makes the web service more secure, reliable, and safer to use.

Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

• Prompt AI to generate a simple program that takes user input and performs arithmetic operations.

• Identify possible runtime errors (e.g., invalid input, division by zero).

• Modify the program to include proper try-except blocks.

• Provide meaningful error messages for invalid inputs.

• Test the improved version with valid and invalid test cases.

Prompt:

Write a Python program that takes user input and performs arithmetic operations like addition, subtraction, multiplication, and division. Then improve the program by adding proper exception handling using try-except blocks to handle errors like invalid input and division by zero. Display clear and user-friendly error messages without crashing the program.

Code:

```python
def main():
    while True:
        print("\nMenu:")
        print("1. Addition")
        print("2. Subtraction")
        print("3. Multiplication")
        print("4. Division")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice in ['1', '2', '3', '4']:
            try:
                num1 = float(input("Enter first number: "))
                num2 = float(input("Enter second number: "))

                if choice == '1':
                    result = num1 + num2
                    print(f"Result: {result}")
                elif choice == '2':
                    result = num1 - num2
                    print(f"Result: {result}")
                elif choice == '3':
                    result = num1 * num2
                    print(f"Result: {result}")
                elif choice == '4':
                    if num2 == 0:
                        raise ZeroDivisionError("Cannot divide by zero.")
                    result = num1 / num2
                    print(f"Result: {result}")

            except ValueError:
                print("Invalid input. Please enter numeric values.")
            except ZeroDivisionError as e:
                print(e)

        elif choice == '5':
            print("Exiting program...")
            break
        else:
            print("Invalid choice. Please try again.")
main()
```

Output:

```
PS C:\Users\AdepuTejaswini\AI assist>

Menu:
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 1
Enter first number: -10
Enter second number: -5
Result: -15.0

Menu:
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 2
Enter first number: 0
Enter second number: 15
Result: -15.0

Menu:
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 3
Enter first number: 0
Enter second number: 6
Result: 0.0

Menu:
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 4
Enter first number: 0
Enter second number: 5
Result: 0.0

Menu:
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Exit
Enter your choice: 5
Exiting program...
PS C:\Users\AdepuTejaswini\AI assist>
```

Observation:

The original code does not handle errors and may crash when invalid input is given. The updated code uses try-except blocks to handle errors like wrong

input and division by zero. This makes the program more reliable and user-friendly, as it shows proper messages instead of stopping suddenly.